

Problem AB23. TRIANGLE

 10 sec.  1024 MB

Author: Boris Mihov

The new leadership of the national committee, represented by the coordinators of groups A and B, wishes to maintain a strict syllabus of the given problems for selecting the national team. For this purpose, they will need to give a geometry problem. Moreover – the number of interactive problems given so far are below standard. To fix the crisis in the selection for IOI, Sashka decided to give a problem that is both interactive and geometric. It has the following statement:

The jury has hidden a permutation $P_0, P_1, \dots, P_{N-1}$ of $\{1, 2, 3, \dots, N\}$. You must find the permutation. For this purpose, you may ask the jury the following question: “Is it possible to form a triangle with positive area with sides P_A, P_B, P_C ?”

Note that it is known (by the triangle inequality) this is possible if and only if:

$$P_A + P_B > P_C, P_A + P_C > P_B \text{ and } P_B + P_C > P_A.$$

Write a program **triangle**, containing a function **solve**, which will be compiled together with the jury program and will communicate with it by asking questions of the type described above. At the end of its execution, it must determine the permutation.

Implementation details

You should implement the **solve** function:

```
std::vector<int> solve(int N)
```

- N : length of the permutation.

This function will be called T times per test – once per subtest, each with equal N and it should return the hidden permutation for the subtest. In order to do this, your program can call the jury function **query**:

```
bool query(int A, int B, int C)
```

- A, B, C : indices of the sides P_A, P_B, P_C .

The function returns **true**, if a triangle with positive area with sides P_A, P_B, P_C exists and **false**, otherwise.

Constraints

- $N = T = 1\,000$

Subtask

Subtask	Points	Description
1	100	The subtask consists of 1 test with $T = 1\,000$ randomly and uniformly sampled from the set of all possible permutations, each of $N = 1\,000$ elements.

The points for a given subtask are obtained only if all the tests for it are successfully passed.

Scoring

Let $Q_{contestant}$ be the average number of queries that your program makes for one call of `solve` on the single test, and let $Q_{target} = 8770$. Then your score for the problem will be:

$$\begin{cases} 0 & \text{if } Q_{contestant} > 2 \times 10^6, \\ 100 & \text{if } Q_{contestant} \leq Q_{target}, \\ 100 \times \max \left(0.15, 1 - \sqrt{1 - \left(\frac{Q_{target}}{Q_{contestant}} \right)^{0.55}} \right) & \text{if } Q_{contestant} > Q_{target}. \end{cases}$$

Sample interaction

For this sample interaction $N = 4$, $T = 2$.

Contestant action	Jury action
	<code>solve(4)</code>
<code>query(0, 0, 0)</code>	<code>true</code>
<code>query(0, 1, 2)</code>	<code>false</code>
<code>query(0, 1, 3)</code>	<code>false</code>
<code>query(0, 2, 3)</code>	<code>true</code>
<code>query(1, 2, 3)</code>	<code>false</code>
<code>return {3, 1, 2, 4};</code>	
	<code>solve(4)</code>
<code>query(0, 1, 2)</code>	<code>false</code>
<code>query(0, 1, 3)</code>	<code>true</code>
<code>query(0, 2, 3)</code>	<code>false</code>
<code>query(1, 2, 3)</code>	<code>false</code>
<code>return {4, 2, 1, 3};</code>	

Local testing

Input format:

- line 1: three integers T , N , and R - the number of subtests, the size of the permutations, and the execution mode. If $R = 1$, the local grader will generate uniformly random permutations and will expect a number on the second line - S , which will be the seed for its random generator. If $R = 2$, the input continues as follows:
- lines 2 to $(1 + T)$: permutations of $\{1, 2, \dots, N\}$.

Output format:

- line 1: an error message or the average number of queries for the subtests if all permutations are correctly identified.